

Lab 05 — Multi-stage builds and production-grade images

Theory — how to go from a 500 MB image that contains your compiler to a 200 MB image that only contains what runs; and what Buildah does instead of BuildKit.

Objectives

- Understand why an image used to **build** must not be the one that **runs**.
- Write a **multi-stage** build for Spring Boot and for Angular.
- Choose a base image knowingly (Debian/Ubuntu, Alpine, distroless).
- Know what BuildKit (Docker) and Buildah (Podman) bring: cache, secrets, stages.
- Relate image size, attack surface and deployment time.

1. The problem: the build tooling stays in the image

A first naive Dockerfile for a Java API:

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21
WORKDIR /app
COPY . .
RUN mvn package -DskipTests
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

It works. It produces an image of **800 MB to 1 GB** containing: a full JDK (compiler, debugging tools), Maven, the local `~/.m2` repository with hundreds of JARs, your **source code**, the tests, and the final JAR. In production, only the JRE and the JAR are used: about 200 MB.

The consequences are not merely cosmetic:

- **Security.** The source code goes to everyone who has the image. Every embedded tool (compiler, `curl`, `git`, shell) is an additional means of action for an attacker, and one more line in the vulnerability scan report.
- **Cost.** 800 MB transferred at every deployment, to every node, stored in every registry, with retention of previous versions.
- **Time.** Starting a container includes downloading the image if it is absent. On a production incident at 3 a.m., the difference shows.

→ SECURITY

An image's **attack surface** is everything an attacker can use once they have obtained code execution: a shell to explore, `curl` to exfiltrate, a compiler to craft a tool, a package manager to install more. Every absent binary is one more step for them. That is why scanners (Trivy, Gype) count packages, and why "minimal" is not only about megabytes.

2. Multi-stage

A Dockerfile can contain **several** `FROM` s. Each opens a *stage* — an independent build environment. Only the **last** one produces the final image; the others are discarded. And `COPY --`

`from=<stage>` lets you fetch files from a previous stage.

```
# ----- stage 1: build -----
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# ----- stage 2: runtime -----
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/api.jar app.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

The final image contains **a JRE and a JAR**. No Maven, no JDK, no sources, no tests, no `.m2`. Nothing that happened in the `build` stage leaves a trace in it — not even in hidden layers, since those layers are simply not part of the image.

REMEMBER

Multi-stage is also the only truly reliable protection against build secrets: what is copied into a discarded stage does not exist in the final image. Be careful though: `COPY --from=build /app /app` would copy everything, secrets included. You copy only the artefact.

The same pattern for Angular:

```
FROM docker.io/library/node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build # produces dist/

FROM docker.io/library/nginx:alpine
COPY --from=build /app/dist/my-app/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

→ ANGULAR

`ng build` compiles the TypeScript components, bundles everything into a few `.js`, `.css` files and an `index.html`, minifies them and names them with a fingerprint (`main-a1b2c3.js`) for browser caching. The result is **static**: any file server serves it. `ng serve`, on the other hand, is a development server that recompiles at every change — precious on your workstation, pointless in production.

Crucial point: **Node does not survive** the build. An Angular front end in production is only HTML, CSS and JavaScript; it needs no server-side JavaScript engine. You **never** containerise `ng serve`.

3. Choosing a base image

Base	Size	Advantages	Drawbacks
debian / ubuntu	75-120 MB	Everything works, full tooling, glibc	Heavy, many packages so many CVEs
*-slim	30-80 MB	Good compromise, still Debian	Fewer tools installed
alpine	5-10 MB	Very light, efficient apk	Uses musl rather than glibc
distroless	20-50 MB	No shell, no package manager	Hard to debug, no exec sh

→ LINUX

The **C library** (`libc`) is the layer between programs and the kernel: `printf` , `malloc` , DNS resolution, locales. Almost every Linux binary depends on it. `glibc` (GNU) is the historical, rich, compatible implementation; `musl` is a minimalist rewrite, chosen by Alpine for its size. A binary compiled for one does not load with the other: `ldd --version` inside the container tells you which one you have.

The Alpine trap deserves a closer look. Most programs cope with `musl` , but not all: native binaries compiled for `glibc` refuse to start, some native Java libraries (compression, cryptography, PDF generation) fail with `UnsatisfiedLinkError` , and differences in DNS resolution or locales appear. Slowdowns have also been measured in the memory allocation of some Java workloads.

In practice, for Spring Boot: `eclipse-temurin:21-jre-alpine` fits the vast majority of cases and halves the size; in case of a native dependency, you go back to `eclipse-temurin:21-jre` (Ubuntu). The choice is **tested**, not decreed.

Distroless images (Google) only contain the runtime and your application: no shell, no `ls` , no package manager. The attack surface is minimal, but `podman exec -it container sh` no longer works — you must have planned your observability otherwise.

4. What really weighs

Four levers, in decreasing order of effectiveness:

1. **Multi-stage** — removes the build tooling. The major lever: 800 MB → 200 MB.
2. **The base image** — `-jre` instead of `-jdk` , `alpine` instead of `ubuntu` .
3. `.dockerignore` — avoids shipping `.git` , `node_modules` , `target` .
4. **Grouping install/clean-up** in the same `RUN` .

Conversely, what has **no** effect: deleting files in a later layer. They stay in the image (lab 02). And the number of layers, by itself, changes almost nothing about size.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman history my-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head
```

5. BuildKit and Buildah

Docker builds with **BuildKit**; Podman builds with **Buildah**. Both read the same Dockerfile and offer the same useful features:

- **Unused stages are not built.** A stage from which nothing is copied into the final image is skipped — Buildah prints `[2/3]`, `[3/3]` and skips `[1/3]`.
- `--target` builds up to a given stage: `podman build --target build -t api-build .` gives you the image of the compilation stage, to inspect it.
- **Persistent caches.** `RUN --mount=type=cache,target=/root/.m2 mvn package` keeps the Maven repository **between builds**, without including it in the image. On a CI agent, the gain is spectacular.
- **Secrets.** `RUN --mount=type=secret,id=npmcrc ...` makes a file available during a single instruction, without ever writing it to a layer (lab 08).

```
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 mvn -q package -DskipTests
```

PODMAN

A visible difference: BuildKit builds independent stages **in parallel**, Buildah builds them one after the other. Another: the `# syntax=docker/dockerfile:1` line, which enables the extended syntax in Docker, is simply **ignored** by Buildah — the `--mount` s work without it. And the `type=cache` cache lives in your user storage (`~/.local/share/containers/storage`), not in a daemon: two users of the same CI server do not share it.

6. Spring Boot: the layers of the JAR

A Spring Boot JAR weighs 50 MB, of which 45 MB are dependencies that almost never change and 5 MB of code that changes at every commit. Copied as a block, it forms a single 50 MB layer retransferred in full at every deployment. Spring Boot knows how to split itself:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine AS extract
WORKDIR /app
COPY target/api.jar api.jar
RUN java -Djarmode=tools -jar api.jar extract --layers --destination extracted

FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=extract /app/extracted/dependencies/ ./
COPY --from=extract /app/extracted/spring-boot-loader/ ./
COPY --from=extract /app/extracted/snapshot-dependencies/ ./
COPY --from=extract /app/extracted/application/ ./
ENTRYPOINT ["java", "-jar", "app.jar"]
```

The dependencies form a stable layer, the code a small volatile layer: deployment only transfers a few MB. Remember the **principle**, which is that of lab 04 applied to the content of a JAR.

7. In the workplace

- **One Dockerfile per service**, multi-stage, versioned with the code. CI needs neither Maven nor Node: `podman build` (or `docker build`) is enough, which guarantees that the CI build and

the workstation build are identical.

- **Tests** often run in a dedicated stage (`RUN mvn test`), so that a red test fails the image build.
- **Vulnerability scanning** (Trivy, Gype) applies to the final image. A minimal image yields a short report, hence one actually acted upon — a 1 GB image yields 300 CVEs nobody will read.
- **The final image runs as non-root**, on a port > 1024, without a shell if possible.

Remember

- What builds must not run: that is the whole point of multi-stage.
- Several `FROM`s = several stages; only the last becomes the image, `COPY --from` fetches the artefact into it.
- Node has no place in the final image of an Angular front end: static content is served by nginx.
- `-jre` rather than `-jdk`, `alpine` if native dependencies allow it, distroless if you accept losing the shell.
- Alpine uses `musl`, not `glibc`: validate with a test, never on principle.
- BuildKit and Buildah offer `--target`, persistent caches and build secrets; Buildah ignores `#syntax=` and does not parallelise.
- Deleting a file in a later layer does not reduce the image size.

Vocabulary

stage: build step opened by a `FROM`. — `COPY --from`: fetching files from another stage or another image. — `--target`: stop the build at a given stage. — **distroless**: image without shell or package manager. — **musl / glibc**: two implementations of the C library. — **BuildKit / Buildah**: Docker's and Podman's build engines. — **cache mount**: persistent cache between builds, outside the image. — **attack surface**: the set of exploitable components present in the image.